9-hybrid-model-2

June 19, 2026

1 Hybrid Model V2

Analysing which traits to pass to an embedding layer so the model can learn if certain chips are "weaker" or "stronger".

1.1 Research

According to literature regarding the physics of printhead behaviour, a printhead chip is governed by distinct physical domains that influence printhead behaviour:

- **Target Electrical Energy**
  - The primary voltage peak that determines the physical extension or deflection of the actuator (piezoelectric or thermal resistor)
  - The dataset provides this as the Voltage (1) parameter
- **Acoustic Waveform Shape**
  - The timing, frequency, and flanks of the electrical pulse. In fluid mechanics, this directly controls the resonance, acoustic wave damping, and oscillation inside the ink channels
  - Dataset provides this as flank ratio
- **Manufacturing Dimensions**
  - The literature shows that pressure chamber and restrictor dimensions strongly influence fluidic capacitance, inductance, resistance, and resonance behavior. Since these geometric properties are fixed after manufacturing, unmeasured geometric variation may contribute to chip-specific behavior.
  - Unmeasured
- **Thermal Environment**
  - Temperature affects ink viscosity and acoustic damping, which in turn influence droplet formation and waveform response. These thermal effects can vary spatially across a printhead assembly.
  - Unmeasured
- **Spatial Assembly Alignment**
  - Industrial printhead assembly literature details the mechanical alignment tolerances required when stitching multiple chips into a long page-wide array. Sub-micron structural misalignments result in systematic, static trajectory errors unique to that chip block's physical placement.
  - Unmeasured
- **True Electrical Efficiency**
  - Studies on integrated printhead electronics highlight electrical crosstalk and trace impedance. The voltage delivered to a chip's internal power rail suffers from minute, steady-state drop-offs compared to the theoretical commanded voltage (Vcap V).

– Unmeasured

The fundamental framework for mapping these variables mathematically traces back to academic research on printhead behavior, notably "The dynamics of the piezo inkjet printhead operation", which outlines how an electrical input transforms into a mechanical deformation, which dictates channel acoustics and subsequent fluid drop formation.

1.2 Basis

I will be using the function defined to represent the system as a guide for model construction.

$$f : \mathbb{R}^{34} \times \mathbb{R}^{33600\times 3} \to \mathbb{R}^{33600}$$

$$f(\theta, U)_i = s(U_i) + \sum_{j\in\mathcal{N}(i)} W_{ij}(\theta), s(U_j) + \varepsilon_i$$

Variables:

Symbol	Meaning	Space
θ	global settings (colour, coverage)	$\mathbb{R}^{34}$
U	per-nozzle input field	$\mathbb{R}^{33600\times 3}$
$U_i = (V_{h(i)}, Fr_{h(i)}, dt2_i)$	input vector for nozzle i	$\mathbb{R}^3$
$s(U_i)$	baseline response of nozzle i	$s : \mathbb{R}^3 \to \mathbb{R}$
$\mathcal{N}(i)$	spatial neighbours of nozzle i	to be defined
$W_{ij}(\theta)$	coupling weight from nozzle j to i	to be defined
y_i	measured print response at nozzle i	$\mathbb{R}$
ε_i	noise	$\mathbb{R}$

1.3 Model

[ ]:
```
import polars as pl
import matplotlib as plt
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np

PARQUET_PATH = "../../data/interim/encoded-collapsed-waveform.parquet"

df = pl.read_parquet(PARQUET_PATH)
```

[ ]:
```
feature_cols = [c for c in df.columns if not c.startswith("Value")]
value_cols   = [c for c in df.columns if c.startswith("Value_")]

X = df[feature_cols].to_numpy()
Y = df[value_cols].to_numpy()

print("df shape:", df.shape)
print("X shape:", X.shape)
print("y shape:", Y.shape)
print("value_cols:", len(value_cols))
print(df[value_cols].shape)
```

1.3.1 Step 1: Data Split

We will split by $(V, F_r, dt2)$ instead of by individual samples, so all measurements belonging to a given $(V, F_r, dt2)$ configuration go entirely into either train or test.

[ ]:
```
configs = df.select(['V', 'F_r', 'dt2']).unique().to_pandas()

train_configs, temp_configs = train_test_split(
    configs,
    test_size=0.3,
    random_state=42
)

val_configs, test_configs = train_test_split(
    temp_configs,
    test_size=0.5,
    random_state=42
)

train_configs['split'] = 'train'
val_configs['split'] = 'val'
test_configs['split'] = 'test'

split_map = pd.concat([train_configs, val_configs, test_configs])

df_pd = df.to_pandas()

df_pd = df_pd.merge(split_map, on=['V', 'F_r', 'dt2'], how='left')

train_df = df_pd[df_pd['split'] == 'train']
val_df   = df_pd[df_pd['split'] == 'val']
test_df  = df_pd[df_pd['split'] == 'test']
```

[ ]:
```
#TRAIN
X_train_t = torch.tensor(train_df[feature_cols].to_numpy(), dtype=torch.float32)
```

```
y_train_t = torch.tensor(train_df[value_cols].to_numpy(), dtype=torch.float32)

# VAL
X_val_t = torch.tensor(val_df[feature_cols].to_numpy(), dtype=torch.float32)
y_val_t = torch.tensor(val_df[value_cols].to_numpy(), dtype=torch.float32)

# TEST
X_test_t = torch.tensor(test_df[feature_cols].to_numpy(), dtype=torch.float32)
y_test_t = torch.tensor(test_df[value_cols].to_numpy(), dtype=torch.float32)

print(f"Train tensors:\n{X_train_t.shape}\n{y_train_t.shape}\n")
print(f"Validation tensors:\n{X_val_t.shape}\n{y_val_t.shape}\n")
print(f"Test tensors:\n{X_test_t.shape}\n{y_test_t.shape}\n")

print(f"Split counts:\n{df_pd['split'].value_counts()}")

print(X_train_t.min(), X_train_t.max())
print(y_train_t.min(), y_train_t.max())
```

[ ]:

1.3.2 Step 2: MLP for Baseline Response

The $(\mathbb{s}(\mathbb{U}_i))$ function maps a 3-dimensional vector per nozzle to a scalar baseline output. Because this baseline doesn't care about neighbors yet, we can pass the entire $(\mathbb{U})$ matrix through a small Multi-Layer Perceptron (MLP) shared across all nozzles.

We will try and compare different dimensional feature spaces to see which one performs better, from a set of (4, 8, 16, 32, 64).

[ ]:
```
train_ds = TensorDataset(X_train_t, y_train_t)
val_ds   = TensorDataset(X_val_t, y_val_t)
test_ds  = TensorDataset(X_test_t, y_test_t)

train_loader = DataLoader(train_ds, batch_size=256, shuffle=True)
val_loader   = DataLoader(val_ds, batch_size=256, shuffle=False)
test_loader  = DataLoader(test_ds, batch_size=256, shuffle=False)
```

[ ]:
```
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        # Learns s(U_i) from physical parameters
        self.mlp = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1120)  # Outputs s(U_i)
        )
```

```
    def forward(self, U):
        return self.mlp(U)
```

[ ]:
```
def train_model(model, train_loader, val_loader, device, epochs=20, lr=1e-3):
    model.to(device)

    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    train_losses = []
    val_losses = []

    for epoch in range(epochs):
        model.train()
        total_train_loss = 0.0
        n = 0

        for x, y in train_loader:
            x, y = x.to(device), y.to(device)

            preds = model(x)
            loss = criterion(preds, y)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_train_loss += loss.item() * x.size(0)
            n += x.size(0)

        train_losses.append(total_train_loss / n)

        # validation
        model.eval()
        with torch.no_grad():
            total_val_loss = 0.0
            n_val = 0

            for x, y in val_loader:
                x, y = x.to(device), y.to(device)
                preds = model(x)
                loss = criterion(preds, y)

                total_val_loss += loss.item() * x.size(0)
                n_val += x.size(0)

        val_losses.append(total_val_loss / n_val)
```

```
    return model, train_losses, val_losses
```

[ ]:
```
def evaluate_model(model, loader, device):
    model.eval()
    criterion = nn.MSELoss(reduction="sum")

    total_loss = 0.0
    total_count = 0

    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)

            preds = model(x)
            loss = criterion(preds, y)

            total_loss += loss.item()
            total_count += y.numel()

    return total_loss / total_count
```

[ ]:
```
import random

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

def compare_models(models_dict, train_loader, test_loader, device, runs=10):
    results = {}

    for name, model_fn in models_dict.items():
        scores = []

        for i in range(runs):
            set_seed(1000 + i)

            model = model_fn().to(device)

            train_model(model, train_loader, device, epochs=20)
```

```
            score = evaluate_model(model, test_loader, device)
            scores.append(score)

        scores = np.array(scores)

        results[name] = {
            "mean": scores.mean(),
            "std": scores.std(),
            "best": scores.min(),
            "all_scores": scores
        }

        print(f"{name}")
        print(f"  mean MSE: {scores.mean():.6f}")
        print(f"  std  MSE: {scores.std():.6f}")
        print(f"  best MSE: {scores.min():.6f}")
        print()

    ranked = sorted(results.items(), key=lambda x: x[1]["mean"])

    print("\n=== FINAL RANKING (by mean MSE) ===")
    for i, (name, stats) in enumerate(ranked):
        print(f"{i+1}. {name} | mean={stats['mean']:.6f} ± {stats['std']:.6f}")

    return ranked[0][0], results
```

[ ]:
```
import time
import torch

input_dim = train_ds.tensors[0].shape[1]

start = time.time()

model = MLP(input_dim=input_dim, hidden_dim=4)

train_model(model, train_loader, val_loader, device, epochs=1)

print("1 epoch time:", time.time() - start)
```

[ ]:
```
input_dim = train_ds.tensors[0].shape[1]

models = {
    "mlp_4": lambda: MLP(input_dim, 4),
    "mlp_8": lambda: MLP(input_dim, 8),
    "mlp_16": lambda: MLP(input_dim, 16),
    "mlp_32": lambda: MLP(input_dim, 32),
    "mlp_64": lambda: MLP(input_dim, 64),
```

```
}

best_model, results = compare_models(
    models,
    train_loader,
    test_loader,
    device
)
```

[ ]:
```
model = MLP(input_dim, 64)

model, train_hist, val_hist = train_model(
    model,
    train_loader,
    val_loader,
    device="mps",
    epochs=50
)
```

According to the test MSE results, 8 dimensions performs better:

- mlp_8: test MSE = 0.155130
- mlp_16: test MSE = 0.467551
- mlp_32: test MSE = 1.753662
- mlp_64: test MSE = 0.217666
- Best model: mlp_8
- Best test MSE: 0.15512989761262405

1.3.3 Step 3: Defining the Coupling Weights $(\mathbb{N}(i))$ and $(W_{ij}(\theta))$

Let's define $\mathcal{N}(i)$ as a local window of radius K (e.g., $K = 2$ means 2 neighbors to the left, 2 to the right).

According to the formula, the coupling weight W depends on θ (global settings). We will build a Hypernetwork that takes θ and generates the interference kernel weights dynamically.

[ ]:
```
import numpy as np

print(int(np.sqrt(1120)))
```

Since the data structure is almost grid-like but uncertain, we will try using a 1D CNN first.

1D CNN

[ ]:
```
class CNN1DModel(nn.Module):
    def __init__(self, input_dim=38, hidden=128, length=1120):
        super().__init__()
```

```
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden),
            nn.ReLU(),
            nn.Linear(hidden, length)
        )

        self.refine = nn.Sequential(
            nn.Conv1d(1, 16, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.Conv1d(16, 16, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.Conv1d(16, 1, kernel_size=5, padding=2)
        )

    def forward(self, x):
        B = x.shape[0]

        y = self.encoder(x)          # (B, 1120)
        y = y.unsqueeze(1)           # (B, 1, 1120)

        y = self.refine(y)           # (B, 1, 1120)

        return y.squeeze(1)
```

2D CNN

[ ]:
```
from torch.utils.data import TensorDataset, DataLoader

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")

X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32)

X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32)
```

[ ]:
```
# ---------------------------
# 2. GLOBAL FEATURES (theta)
# ---------------------------

theta_train = torch.tensor(
    train_df[theta_cols].to_numpy(),
    dtype=torch.float32,
    device=device
)

theta_test = torch.tensor(
    test_df[theta_cols].to_numpy(),
```

```
    dtype=torch.float32,
    device=device
)
```

[ ]:
```
# --------------------------
# 3. LOCAL FEATURES (U)
# --------------------------

U_train = torch.tensor(
    train_df[local_cols].to_numpy().reshape(-1, 33600, 3),
    dtype=torch.float32,
    device=device
)

U_test = torch.tensor(
    test_df[local_cols].to_numpy().reshape(-1, 33600, 3),
    dtype=torch.float32,
    device=device
)
```

[ ]:
```
import torch.nn.functional as F

class CouplingNetwork(nn.Module):
    def __init__(self, neighbor_radius=2):
        super().__init__()
        self.radius = neighbor_radius
        self.kernel_size = 2 * neighbor_radius + 1

        # theta -> spatial interaction kernel
        self.weight_generator = nn.Sequential(
            nn.Linear(34, 32),
            nn.ReLU(),
            nn.Linear(32, self.kernel_size)
        )

        # pre-build center mask (no self-interaction)
        mask = torch.ones(self.kernel_size)
        mask[self.radius] = 0.0
        self.register_buffer("mask", mask)

    def forward(self, theta, baseline_s):
        """
        theta: (B, 34)
        baseline_s: (B, N, 1)
        """

        B, N, _ = baseline_s.shape
```

```
        # 1. generate kernel per sample
        kernel = self.weight_generator(theta)          # (B, K)

        # enforce no self-interaction (safe, differentiable)
        kernel = kernel * self.mask.unsqueeze(0)       # (B, K)

        # 2. reshape for grouped conv
        x = baseline_s.transpose(1, 2)                 # (B, 1, N)
        kernel = kernel.view(B, 1, self.kernel_size)   # (B, 1, K)

        # 3. grouped convolution: each sample has its own kernel
        interference = F.conv1d(
            x,
            kernel,
            padding=self.radius,
            groups=B
        )  # (B, 1, N)

        return interference.transpose(1, 2)            # (B, N, 1)
```

Step 4: Combining the System $f(\theta, U)_i$

With 1D CNN

[ ]:
```
import torch
import torch.nn as nn

class PrinterOutputPredictor(nn.Module):
    def __init__(self, input_dim, hidden_dim=128, output_len=1120):
        super().__init__()

        self.output_len = output_len

        # 1. global feature encoder (38D → latent)
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_len),
        )

        # 2. 1D CNN over nozzle axis
        self.refine = nn.Sequential(
            nn.Conv1d(1, 16, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.Conv1d(16, 16, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.Conv1d(16, 1, kernel_size=5, padding=2),
```

```
        )

    def forward(self, x):
        """
        x: (B, 38)
        returns: (B, 1120)
        """

        # 1. project global features into spatial signal
        y = self.encoder(x)              # (B, 1120)

        # 2. treat as 1D spatial field
        y = y.unsqueeze(1)               # (B, 1, 1120)

        # 3. spatial refinement via CNN
        y = self.refine(y)               # (B, 1, 1120)

        return y.squeeze(1)
```

[ ]:
```
from torch.utils.data import TensorDataset, DataLoader

train_dataset = TensorDataset(X_train_t, y_train_t)
test_dataset  = TensorDataset(X_test_t, y_test_t)

train_loader = DataLoader(train_dataset, batch_size=256, shuffle=True)
test_loader  = DataLoader(test_dataset, batch_size=256, shuffle=False)
```

[ ]:
```
import torch
import torch.nn as nn

def train_model(
    model,
    train_loader,
    test_loader,
    epochs=200,
    lr=1e-3,
    device="cpu",
    save_path="cnn1d_checkpoint.pth"
):

    model = model.to(device)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    for epoch in range(epochs):

        # -------------------
```

```
        # TRAIN
        # ------------------
        model.train()
        train_loss_sum = 0.0

        for X_batch, y_batch in train_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            preds = model(X_batch)
            loss = criterion(preds, y_batch)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            train_loss_sum += loss.item()

        # ------------------
        # EVAL
        # ------------------
        if epoch % 10 == 0:

            model.eval()
            with torch.no_grad():

                test_loss_sum = 0.0
                test_mae_sum = 0.0
                n = 0

                for X_batch, y_batch in test_loader:
                    X_batch = X_batch.to(device)
                    y_batch = y_batch.to(device)

                    preds = model(X_batch)
                    loss = criterion(preds, y_batch)

                    test_loss_sum += loss.item() * X_batch.size(0)
                    test_mae_sum += torch.abs(preds - y_batch).mean().item()
                    n += 1
                    test_mae = test_mae_sum / n

            print(
                f"Epoch {epoch:4d} | "
                f"Train Loss {train_loss_sum/len(train_loader):.6f} | "
                f"Test Loss {test_loss_sum/n:.6f} | "
                f"Test MAE {test_mae:.6f}"
```

```
            )

            torch.save({
                "epoch": epoch,
                "model_state_dict": model.state_dict(),
                "optimizer_state_dict": optimizer.state_dict(),
            }, save_path)
```

[ ]:
```
device = torch.device("mps")

train_model(
    PrinterOutputPredictor(input_dim),
    train_loader,
    test_loader,
    epochs=100,
    lr=1e-3,
    device=device
)
```

2D CNN

[ ]:
```
input_dim = X_train.shape[1]
hidden_dim = 8


class PrinterOutputPredictor(nn.Module):
    def __init__(self, input_dim, hidden_dim, neighbor_radius=2):
        super().__init__()

        # local physics / per-nozzle response
        self.baseline_net = MLP(input_dim, hidden_dim)

        # spatial coupling operator
        self.coupling_net = CouplingNetwork(neighbor_radius)

        # learnable scale for coupling strength (important stabilizer)
        self.coupling_scale = nn.Parameter(torch.tensor(0.0))

    def forward(self, theta, U):
        """
        theta: (B, 34)          global / chip-level settings
        U:     (B, N, 3)        per-nozzle features
        """

        # 1. local nonlinear response per nozzle
        baseline_s = self.baseline_net(U)  # (B, N, 1)

        # 2. spatial interaction term conditioned on global state
```

```
        interference = self.coupling_net(theta, baseline_s)  # (B, N, 1)

        # 3. learned mixing strength (prevents coupling dominance early)
        predicted_y = baseline_s + torch.exp(self.coupling_scale) * interference

        return predicted_y.squeeze(-1)
```

[ ]:
```
def train_model(
    model,
    X_train_t,
    y_train_t,
    X_test_t,
    y_test_t,
    epochs=200,
    lr=1e-3,
    device="cpu",
    save_path="models/checkpoint.pth"
):
    model = model.to(device)

    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    X_train_t = X_train_t.to(device)
    y_train_t = y_train_t.to(device)
    X_test_t  = X_test_t.to(device)
    y_test_t  = y_test_t.to(device)

    for epoch in range(epochs):
        model.train()

        preds = model(X_train_t)
        loss = criterion(preds, y_train_t)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if epoch % 10 == 0:
            # save checkpoint
            torch.save({
                "epoch": epoch,
                "model_state_dict": model.state_dict(),
                "optimizer_state_dict": optimizer.state_dict(),
                "loss": loss.item(),
            }, save_path)
```

```
            # evaluation mode
            model.eval()
            with torch.no_grad():

                train_preds = model(X_train_t)
                train_loss = criterion(train_preds, y_train_t).item()
                train_mae = torch.mean(torch.abs(train_preds - y_train_t)).
 ↪item()

                test_preds = model(X_test_t)
                test_loss = criterion(test_preds, y_test_t).item()
                test_mae = torch.mean(torch.abs(test_preds - y_test_t)).item()

            print(
                f"Epoch {epoch:4d} | "
                f"Train Loss: {train_loss:.6f} | Test Loss: {test_loss:.6f} | "
                f"Train MAE: {train_mae:.6f} | Test MAE: {test_mae:.6f}"
            )
```

[ ]:
```
device = "mps"

model = PrinterOutputPredictor(input_dim, hidden_dim, neighbor_radius=2)

train_model(
    model,
    X_train_t,
    y_train_t,
    X_test_t,
    y_test_t,
    epochs=200,
    lr=1e-3,
    device=device,
    save_path="models/printer_model.pth"
)
```